色キューブ仕分けタスクにおける ACT 方策への SAE 介入

Physical AI 応用編1 最終課題　説明資料

1. 概要と目的

ロボットの模倣学習方策の内部表現に介入して、方策の行動を後から変えられるかを調べた。

- **タスク**: 机の上に色の違う4つの箱（赤・緑・青・黄）が固定位置にあり、4色のうち1色のキューブが1つ置かれている。Panda アームがキューブを掴み、キューブと同じ色の箱に入れる。
- **方策**: ACT（Action Chunking with Transformers）を、スクリプト方策で生成したデモから模倣学習した。
- **目的**: 学習済みの重みは固定したまま、推論時に内部表現へ介入するだけで、「赤いキューブを緑・青・黄の箱に入れる」ことができるかを調べる。
- **主な問い**: 「意図レベル」（どの箱へ運ぶか）と「運動レベル」（手先をどちらへ動かすか）の介入で、到達できる範囲と副作用はどう違うか。

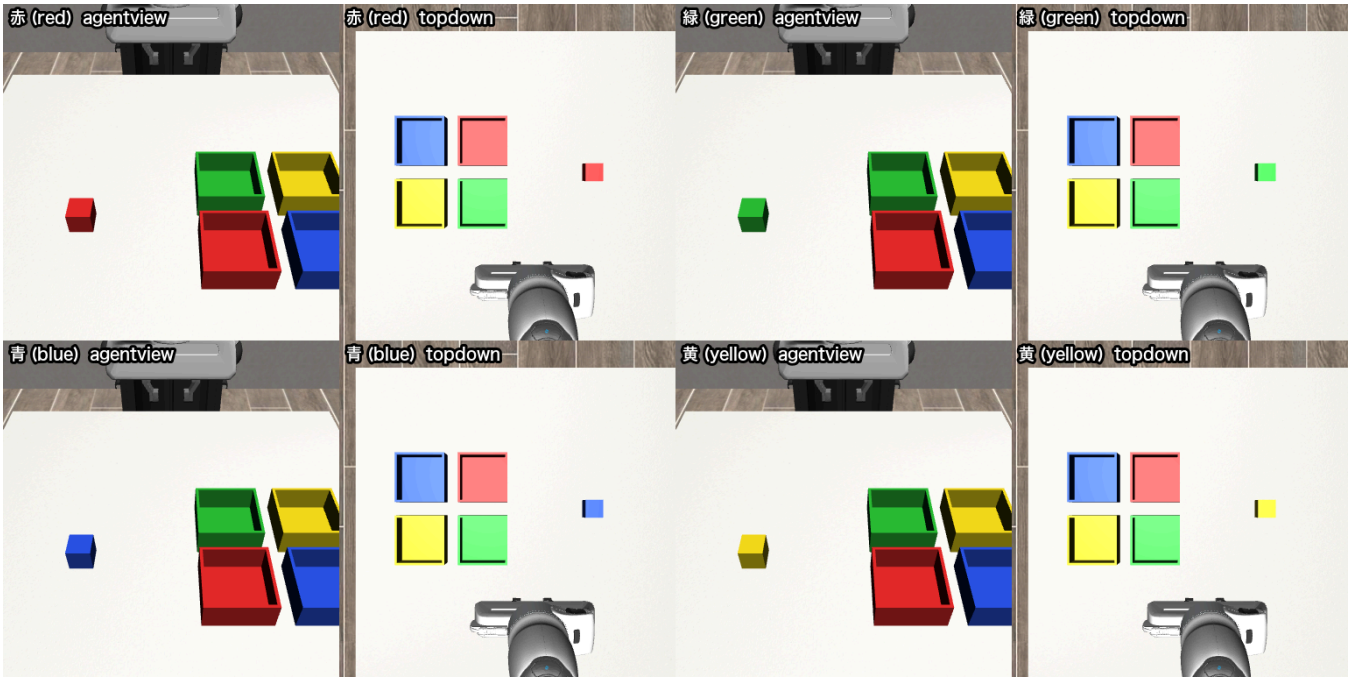
介入の方法は、疎な特徴に分解する TopK SAE（Sparse Autoencoder）と、平均活性の差ベクトルを足す DiffMean の2つを比較した。

2. 環境とデータ

2.1 環境

robosuite 1.5.1（MuJoCo 3.3）上に、`ManipulationEnv` を継承した自作環境 `CubeSortEnv` を作った。

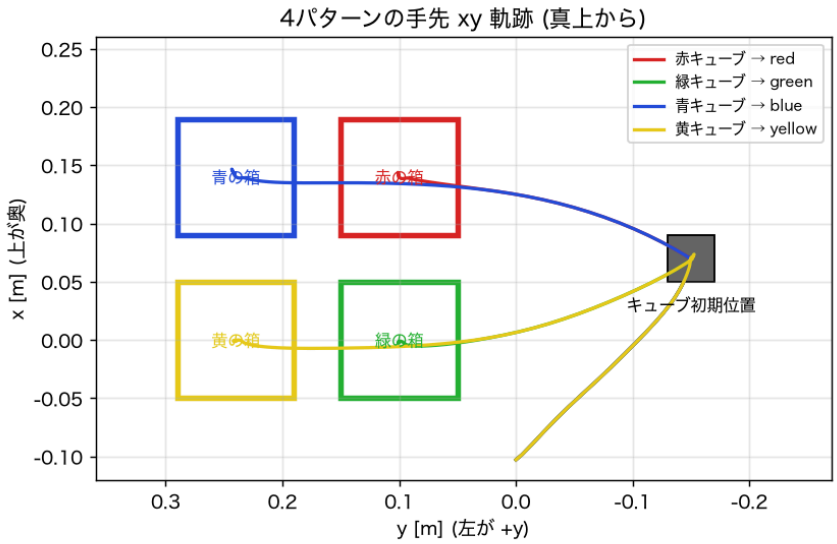
- ロボット: Panda。制御は OSC_POSE の差分指令で、行動は7次元（位置3・回転3・グリッパー1）、20Hz。
- 配置（すべて固定）:
  - キューブ: 右側 (x, y) = (0.07, −0.15)
  - 箱（内寸 10cm）: 左側に 2×2。赤 (0.14, 0.10)、緑 (0.00, 0.10)、青 (0.14, 0.24)、黄 (0.00, 0.24)
  - 赤→緑は x 方向だけ、赤→青は y 方向だけずれる（隣）。赤→黄は x と y の両方（対角）。
- 初期状態のノイズ: キューブの xy ±1cm、向き ±10°、関節角 ±0.02rad。
- 配置はサンプルを確認しながら4回修正した（Panda の可動範囲に合わせる、キューブを右・箱を左にする、不要な手首の回転をなくす、キューブの x を箱の2列の中間にする）。



2.2 データ

ウェイポイント（接近 → 降下 → 把持 → 持ち上げ → 運搬 → 下降 → 離す → 退避）を手先位置の P 制御で追うスクリプト方策で、デモを生成した。

- 色ごとに 100本、合計 400本。新たな人手デモは集めていない。
- デモの多様性: ウェイポイントの xy に ±1cm のノイズ、P ゲインに 0.8〜1.2 倍のばらつき、行動にガウスノイズ（σ=0.05）を加えた。
- 採用率（成功条件を満たした割合）は4色とも 100%。

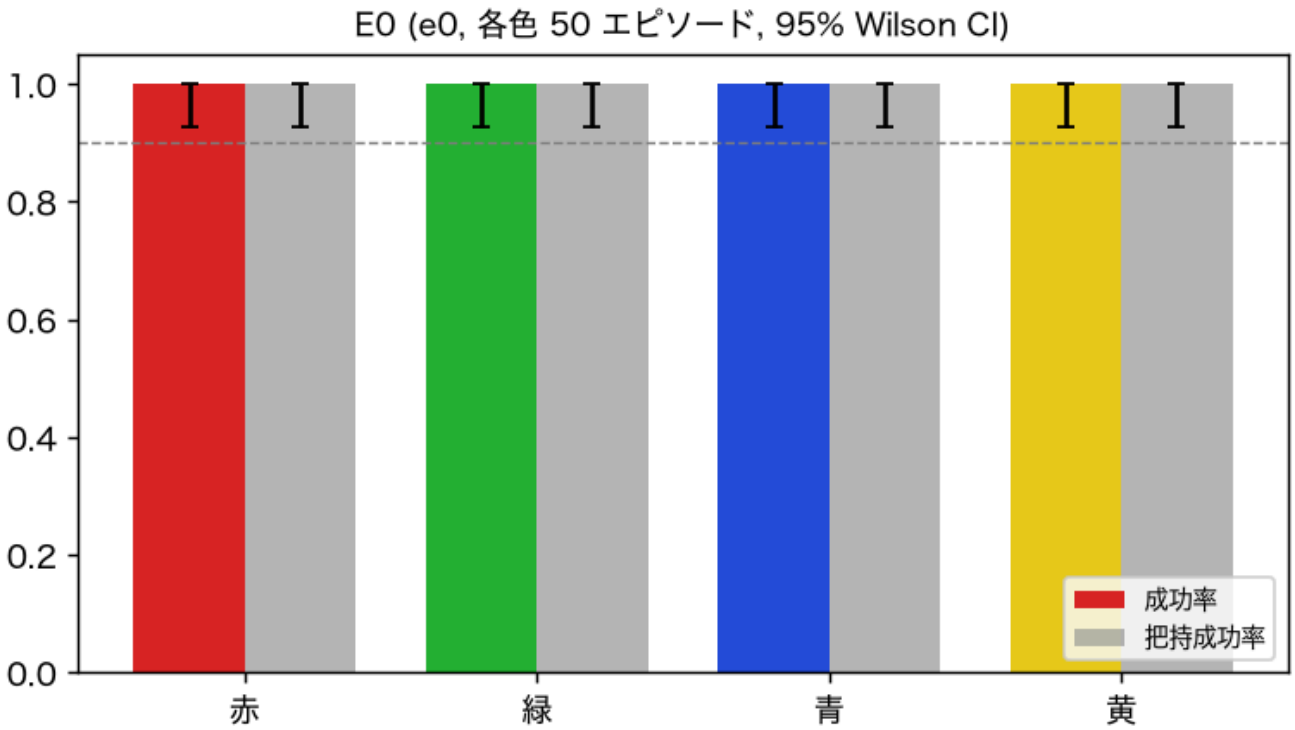


3. 手法

3.1 ACT

最小構成の ACT を自前で実装した。

- 入力は3つのトークン。
  - robot: 手先の位置・姿勢・グリッパーの開き（9次元）
  - cube: キューブの位置・姿勢・手先からの相対位置（10次元）
  - color: キューブの色の one-hot（4次元）
- 構成: チャンク長 20、1回の推論で実行するのは 5ステップ、d_model 256、encoder 4層・decoder 4層、CVAE の潜在 32次元（推論時は z=0）。
- Temporal Ensembling は使っていない。過去のチャンクを混ぜると、介入の効果がなまり、元の動作へ戻る力が強まるため。
- 学習: 30k step（MPS で約25分）。
- **ベースライン（E0）**: 4色とも成功率 100%（各 50エピソード、95% 信頼区間 [93%, 100%]）。



3.2 SAE と介入

- encoder 各層の出力（4か所）と decoder 各層の出力（4か所）に、TopK SAE（辞書 2048、k=32/16）を学習した。color トークンだけで学習した「色の SAE」も作った。
- 介入は誤差を保存する形にした。活性 h を h' = h + W_dec(f' − f) に置き換えるので、SAE が再構成できない成分は壊れない。
- 比較のための DiffMean は、平均活性の差ベクトルを足す。
- 介入は把持後（キューブを持ち上げた後）だけに入れた。介入するタイミングの判定には、シミュレータの真の状態を使っている。

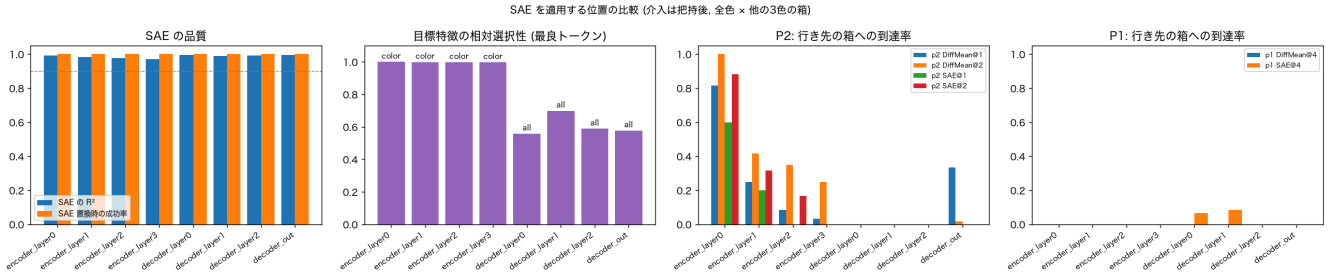
介入は3種類調べた。

介入	位置	SAE	DiffMean
① 色（意図レベル）	encoder の color トークン	元の色の特徴を 0 にし、行き先の色の特徴をその色での平均 × β にする	μ_行き先 − μ_元 を α 倍して足す
② x/y 移動（運動レベル）	decoder 出力	+x/−x/+y/−y の方向の特徴に α を足す	手先が ±x / ±y に動くときの平均活性の差を足す
③ 軌道	decoder 出力	色ごと（＝軌道パターンごと）の特徴を swap する	位置 k ごとの色平均の差を足す

4. 結果

4.1 SAE の品質と、介入する位置の比較

- 8か所すべてで $R^2 \geq 0.97$。SAE で置き換えても成功率は 100% のまま。
- encoder の浅い層は死んだ特徴が 10% を超えた（層0: 30%）。
- 色の介入は encoder の最初の層（color トークン）が最もよく効き、深い層ほど効かなくなった。

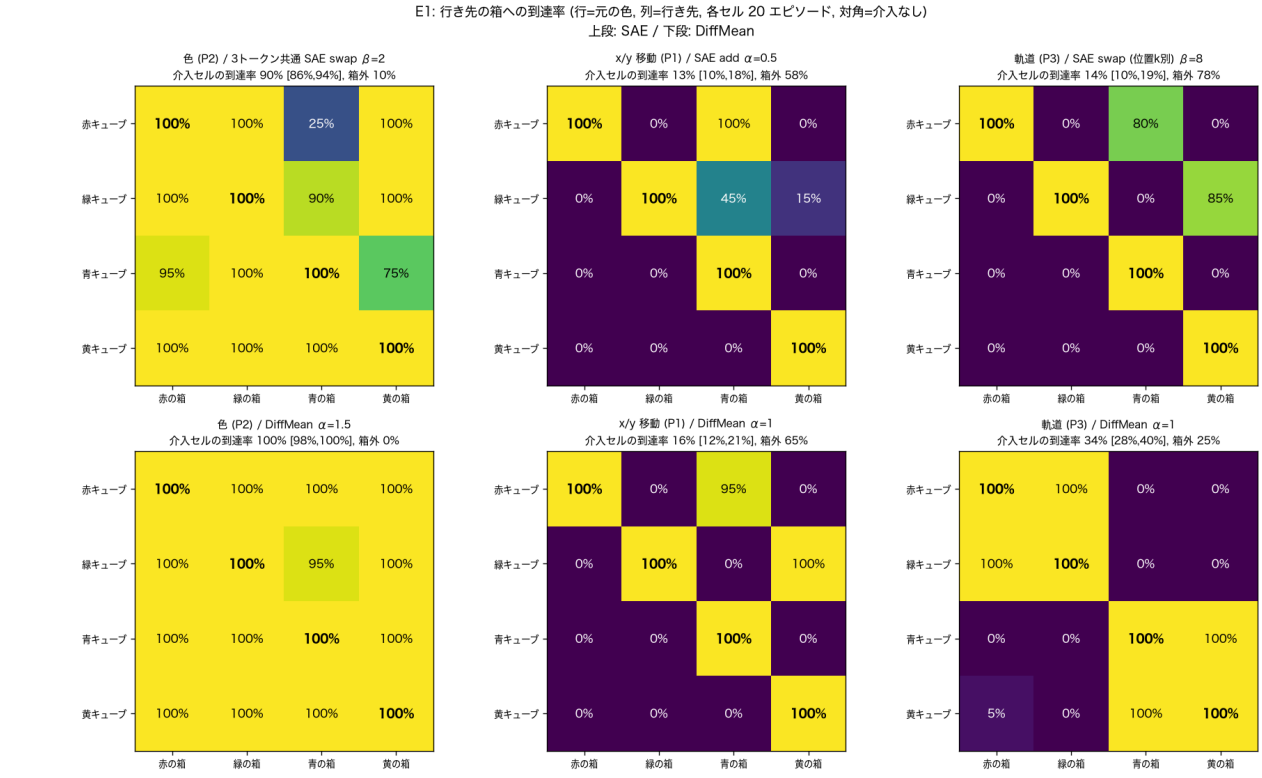


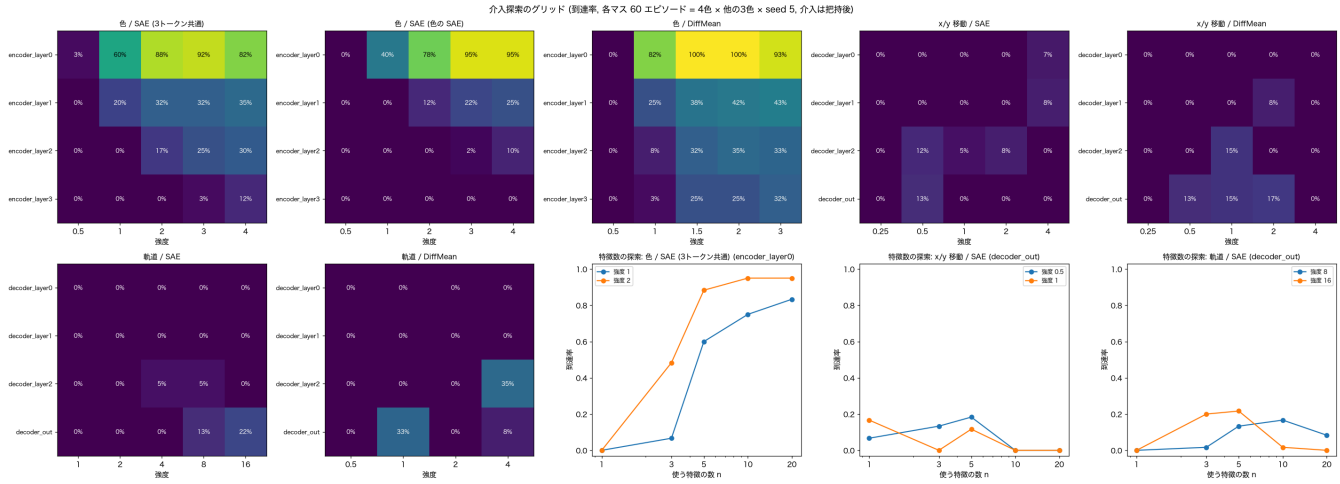
4.2 E1: 介入3種類 × SAE / DiffMean

行き先の箱への到達率（12通り × 20エピソード）。

介入	SAE	DiffMean
① 色	90%（3トークン共通の SAE）/ 94%（色の SAE）	**100%**
② x/y 移動	13%	16%
③ 軌道	14%	34%

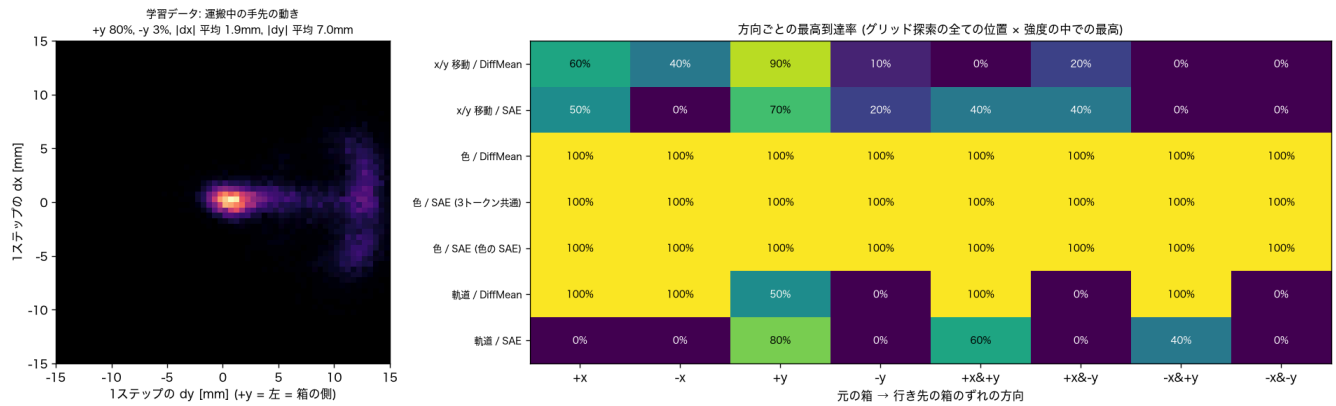
- 色の介入は、エピソードの最初から入れても把持は崩れず、行き先だけが変わった。
- 網羅性: 位置（候補の全層）× 強度（5段階）× 特徴数（1〜20）の 166通り（9960エピソード）を試しても、x/y は最高 17%、軌道は最高 35% だった。



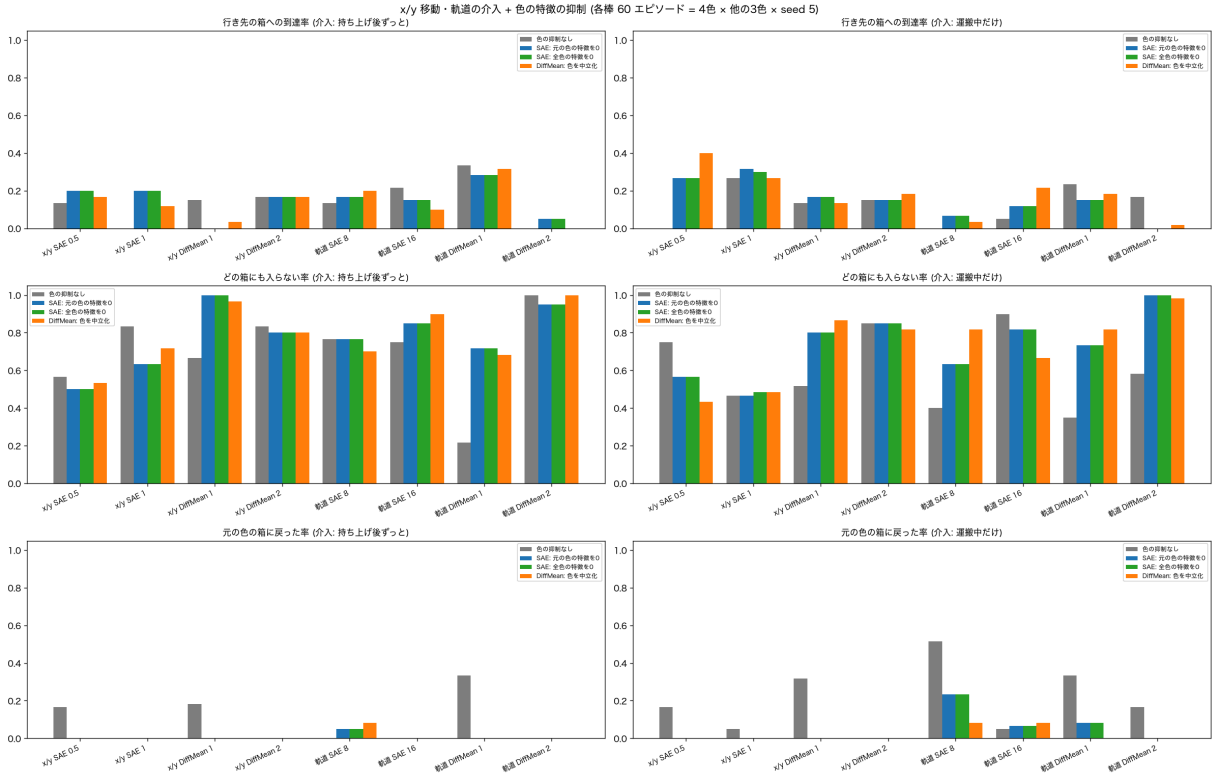


4.3 x/y と軌道が効かなかった要因

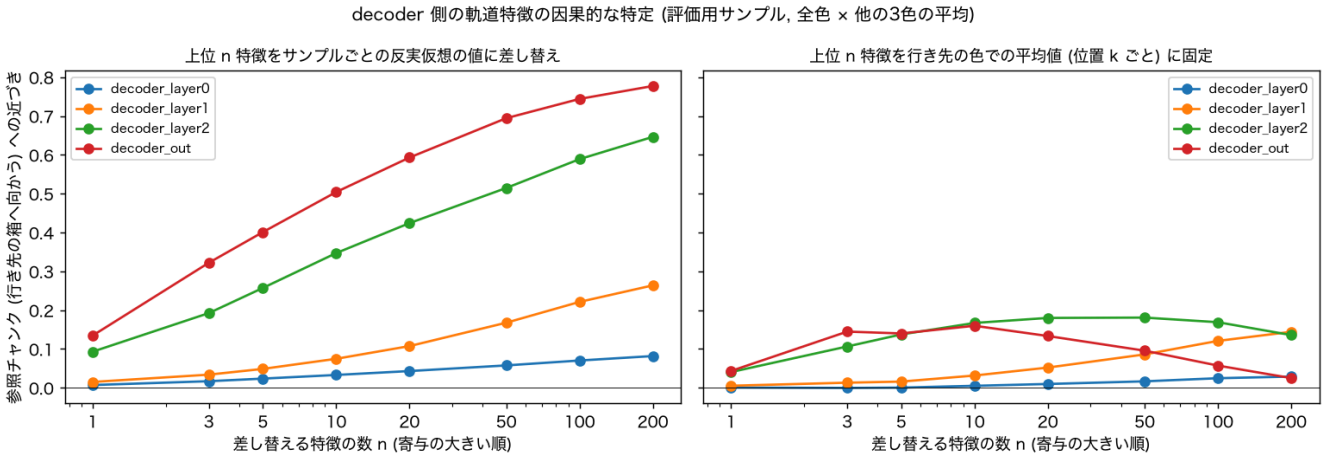
- **データの偏り**: 運搬中の手先の動きは +y が 80%、−y は 3%。x/y と軌道の介入が成功したのは、ほぼ +y 方向だけだった。



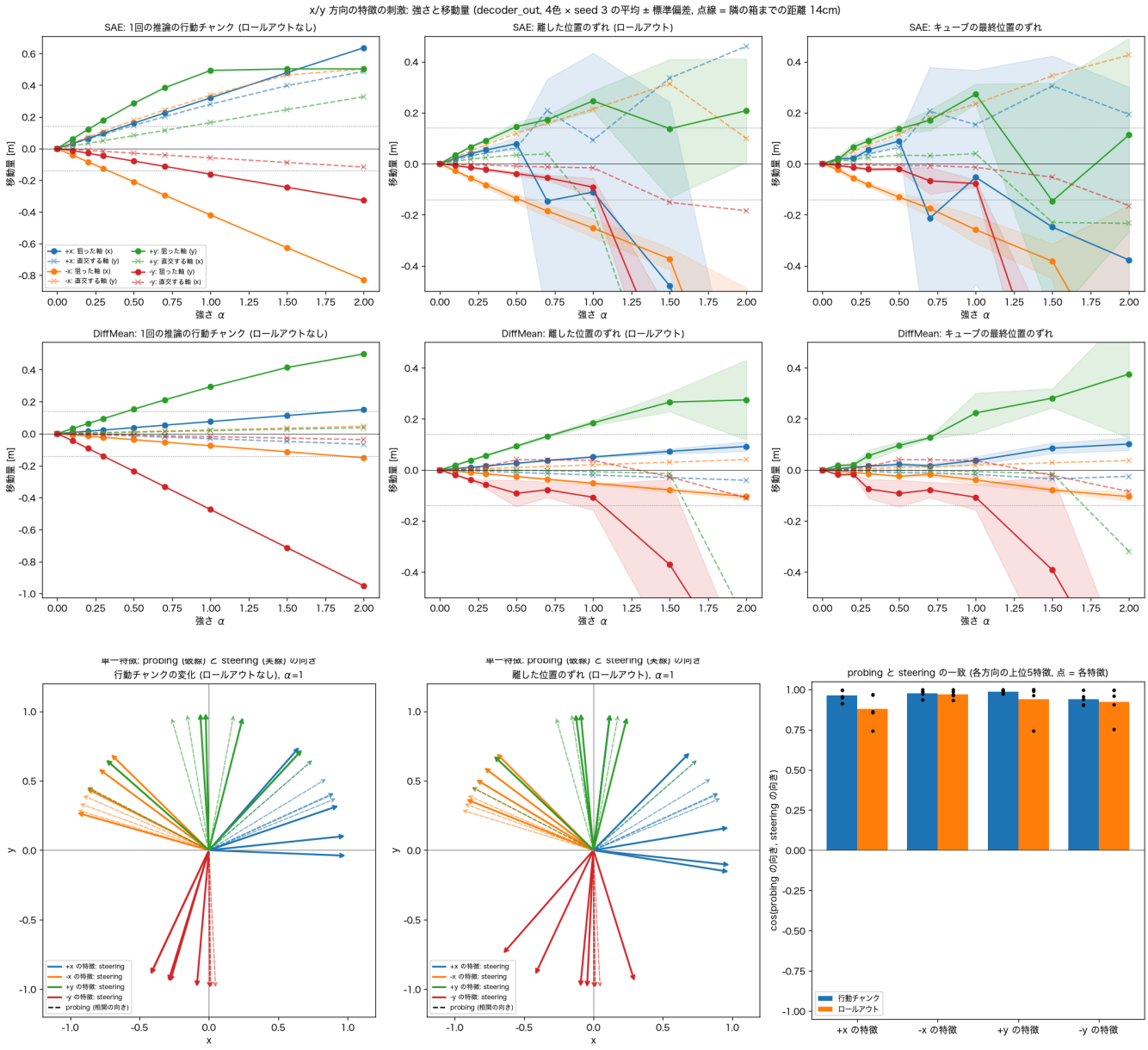
- **元の箱への引き戻し**: 色の特徴を同時に抑えると、元の箱へ戻る率は 14% → 2〜3% に下がった。しかし到達率は上がらず、箱外が増えた。色を抑えるだけでは、キューブはどの箱にも置かれなかった。



- **行動チャンクの直接比較**: color トークンを行き先の色に変えたときに方策が出すチャンクを基準にして、各介入でチャンクがそこにどれだけ近づくかを測った。色の介入は 81〜89%、x/y は 20〜33%（向きは合っているが、状態によらない一定の押し）、軌道の SAE は −1% だった。
- **軌道の特徴**:
  - 行き先の情報は decoder の活性から線形に約 100% 読み出せる。しかし、色ごとに選んだ20特徴では 59〜73% しか読めない。
  - 反実仮想（color だけを変えた実行）との差と勾配で特徴を選び直すと、decoder 出力の約50特徴（2048 中）を状態に応じた値に差し替えれば、軌道が 90% 切り替わった。決まった値に固定すると 2〜8% にとどまる。
  - 4パターンの違いは、関節角度でも内部表現でも「x の行き先 + y の行き先」の足し算でほぼ説明できた（パターン固有の部分は関節角度 0.6%、decoder 4〜13%）。



- **probing と steering**: 1つの特徴について、データ上の相関の向きと刺激したときの動きはよく一致した（cos 0.88〜0.99）。しかし、1個で x 軸に沿って動く特徴はなかった。



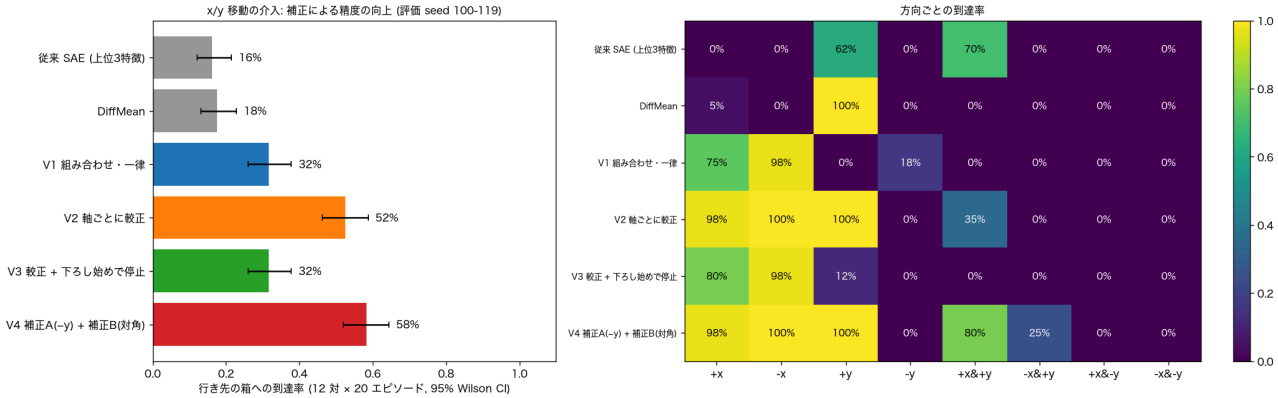
4.4 x/y 移動の介入の改善

- 特徴を組み合わせて選んだ。7次元すべての行動成分について、狙った軸以外が 0 になる組み合わせを NNLS で求めた。
- 軸ごとに強さを較正した。
- 対角は対ごとに強さを決め、−y を含む方向では下ろし始めたら介入を止めた。
- 評価には、調整に使っていない seed を使った。

条件	到達率
従来の SAE（上位3特徴）	16%
DiffMean	18%
組み合わせで選ぶ	32%
+ 軸ごとに較正	52%

条件	到達率
+ 対角の補正・下ろし始めで停止（–y を含む方向のみ）	**58%**
上に色の抑制を追加	36〜48%

- 1軸の移動は +x 98%、–x 100%、+y 100% まで届くようになった。
- –y を含む方向は、どの補正でも 0% だった。方策は運搬の高さで止まったまま、上限の 400ステップまで終わった。



5. 考察

1. **行き先を自在に変えられるのは、目的（色）の表現への介入だけだった。** color トークンを差し替えると、cross-attention を通じて decoder のすべての層の計算が変わり、今の状態に合った「行き先の箱への計画」がそのまま出てくる。
2. **運動レベル（x/y）の介入は、目的が決める「止まる位置」をずらす形でしか働かない。** 方策は自分の目的（元の箱）に着いたと判断したときに下ろし始める。そのため、元の箱を通り過ぎる方向や同じ距離の方向へは補正すれば届く。一方、目的より手前で置く必要がある方向（–y）には届かない。色を抑えると、届く方向と届かない方向が入れ替わるだけだった。
3. **decoder 側に「4パターンの軌道」の記号はない。** 軌道は x と y の成分に分かれ、状態に応じて値が変わる形で表現されている。そのため、決まった値で刺激する介入では軌道全体を切り替えられない。
4. **SAE と DiffMean**: 色の介入では DiffMean（100%）が SAE（90〜94%）を上回った。x/y の介入では、SAE の特徴を組み合わせて使うことで DiffMean（18%）を大きく上回った（58%）。SAE の利点は、行動の成分ごとに分解して、狙った方向だけに効く組み合わせを作れる点にあった。
5. **限界**:
6. 色と行き先は常に1対1で、箱の配置も固定（方策は箱を観測していない）なので、色の特徴が「色の識別」なのか「箱 c 行き」なのかはデータから区別できない。
7. 介入するタイミングの判定に、シミュレータの真の状態を使っている。
8. 2×2 の4パターン、Temporal Ensembling なし、辞書 2048 の TopK SAE という条件での結論である。

6. 工夫した点、うまくいかなかった点と反省

- **工夫した点**

    - 「色 / x/y / 軌道」を SAE と DiffMean で同じ条件で比べ、位置・強度・特徴数を網羅的に探索して、到達率の上限を示した。
    - 効かなかった理由を、データの偏り、行動チャンクの直接比較、反実仮想による因果的な特徴選択、関節角度の分解で切り分けた。
    - x/y 介入の失敗を、特徴の組み合わせ選択（グリッパーなど他の成分の混入の除去を含む）と較正で 16% → 58% に改善し、残る失敗（–y）の理由を示した。
    - 調整と評価で seed を分け、合わせ込みを避けた。
- **うまくいかなかった点と反省**
    - 最初は別の仕様書（MimicGen データ版）で作業を始めてしまい、時間を失った。
    - x/y 特徴を「上位3つを独立に選ぶ」方法で選んでいたため、斜めの特徴が足し合わさり、最初は運動レベルの介入を過小評価した。特徴の組み合わせを選ぶときも、最初は位置しか条件にしておらず、グリッパーの成分が混ざってキューブを落とした。
    - –y（目的より手前で置く方向）は、運動レベルの介入だけでは解決できなかった。方策の「着いたら下ろす」判断に介入する方法（目的の表現と組み合わせるなど）は今後の課題である。
    - 色と行き先を切り分けるには、箱の配置を入れ替える、画像入力にするなどの設定が必要だった。

引用

- Y. Zhu et al., "robosuite: A Modular Simulation Framework and Benchmark for Robot Learning," arXiv:2009.12293.
- T. Z. Zhao, V. Kumar, S. Levine, C. Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware," RSS 2023.
- E. Todorov, T. Erez, Y. Tassa, "MuJoCo: A physics engine for model-based control," IROS 2012.
- L. Gao et al., "Scaling and evaluating sparse autoencoders," arXiv:2406.04093.